

ESP-IDF FreeRTOS Task Fundamentals

Converting a Single Polling Loop into Two Independent, Scheduler-Driven Tasks with ESP_LOG Diagnostics



*Dana Natsheh
Cyber Robot
IoT*

1. Executive Summary

This report documents Task 3.6: converting the previous single `while(1)` polling loop – which read a potentiometer and an IR obstacle sensor and drove two LEDs directly from `app_main` – into two independent FreeRTOS tasks created with `xTaskCreate()`. **Task A (SensorTask)** owns the ADC read of the potentiometer and the digital read of the IR sensor. **Task B (OutputTask)** owns the LEDC PWM duty update and the digital IR-triggered LED, and runs at a strictly higher priority than Task A. The two tasks share their readings through plain global variables – deliberately, and without a queue or mutex – so that the resulting data-sharing hazard can be observed and reasoned about directly, ahead of the queue-based fix planned for Task 3.7. All logging in this project now goes through `ESP_LOGI`, tagged per task, replacing every `printf` call used previously.

2. System Overview & Goal

```
Task A -- SensorTask (priority 3, every 200ms)
(Potentiometer wiper) --adc_oneshot_read--> g_pot_raw
(IR sensor OUT) --gpio_get_level--> g_ir_state

Task B -- OutputTask (priority 4, every 100ms)
g_pot_raw --shift to 10-bit--> [LEDC PWM] --> LED1 brightness
g_ir_state --gpio_set_level--> LED2 on/off
```

- The single polling loop from the previous task is now two independent infinite loops, each living inside its own task function, each paced by its own `vTaskDelayUntil` rather than a shared `vTaskDelay` in `app_main`.
- Task A and Task B run at different periods (200 ms and 100 ms respectively) and different priorities, so `OutputTask` applies each freshly-read potentiometer value roughly twice before `SensorTask` refreshes it again.
- Data crosses from Task A to Task B purely through three `volatile` globals (`g_pot_raw`, `g_pwm_duty`, `g_ir_state`) – no queue, semaphore, or mutex protects this hand-off in this task.
- `app_main` itself becomes a third, low-priority housekeeping loop: it creates both tasks once, then periodically logs runtime statistics and does nothing else.

3. Hardware Architecture

Pin	Peripheral	Role
GPIO2	LEDC PWM channel 0, timer 0 (10-bit, 5 kHz)	Brightness LED, duty written by OutputTask
GPIO4	Digital output	Second LED, mirrors the IR sensor state as written by OutputTask
GPIO5	Digital input, internal pull-up enabled	IR obstacle sensor output, read by SensorTask ; active-low (object present when the pin reads 0)
GPIO34	ADC1 channel 6	Potentiometer wiper; input-only pin, read by SensorTask

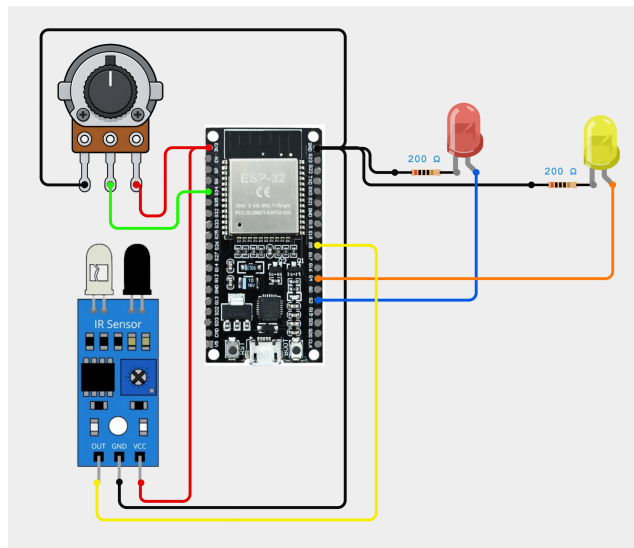


Figure 1: Wiring schematic (unchanged from the previous single-loop task).

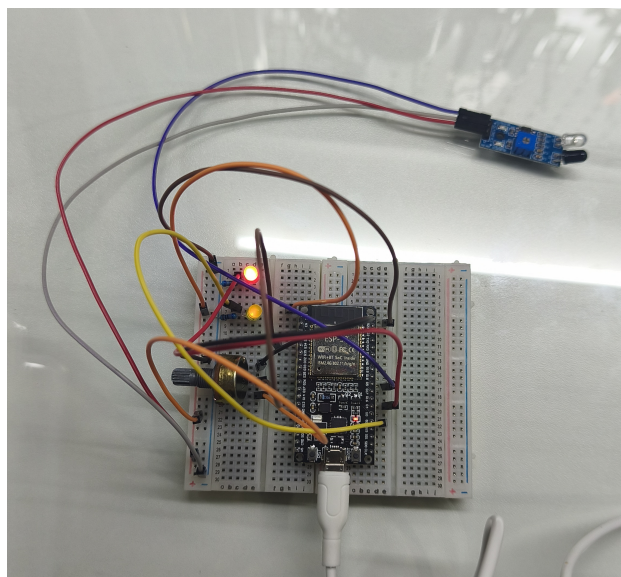


Figure 2: Assembled breadboard circuit.

Power wiring is unchanged from the previous task: every module's VCC/+ leg returns to the ESP32's 3V3 rail and every GND/− leg returns to GND, and both LEDs sit in series with a 220–330 Ω current-limiting resistor. The only functional hardware-side change in this task is enabling the ESP32's internal pull-up on the IR sensor's input pin in software, rather than relying on the sensor module's own output stage alone.

4. Development Environment & Toolchain Setup

- Target ESP-IDF version: v5.0 or later, required by the `adc_oneshot` driver carried over from the previous task.
- Build system: PlatformIO (`platform = espressif32`, `framework = espidf`, `board = esp32dev`), the same toolchain reached at the end of the previous task's setup journey. No further toolchain migration was needed for this task.

```
1 [env:esp32dev]
2 platform = espressif32
3 board = esp32dev
4 framework = espidf
5 monitor_speed = 115200
```

Listing 1: platformio.ini

5. Firmware Implementation

Peripheral initialization

```
1 static void init_peripherals(void)
2 {
3     adc_oneshot_unit_init_cfg_t init_cfg = { .unit_id = POT_ADC_UNIT };
4     adc_oneshot_new_unit(&init_cfg, &adc_handle);
5
6     adc_oneshot_chan_cfg_t chan_cfg = {
7         .bitwidth = ADC_BITWIDTH_DEFAULT,
8         .atten     = ADC_ATTEN_DB_12,
9     };
10    adc_oneshot_config_channel(adc_handle, POT_ADC_CHANNEL, &chan_cfg);
11
12    gpio_config_t ir_in_cfg = {
13        .pin_bit_mask = 1ULL << IR_SENSOR_GPIO,
14        .mode = GPIO_MODE_INPUT,
15        .pull_up_en = GPIO_PULLUP_ENABLE,
16    };
17    gpio_config(&ir_in_cfg);
18 }
```

```

19  gpio_config_t ir_led_cfg = {
20      .pin_bit_mask = 1ULL << IR_LED_GPIO,
21      .mode = GPIO_MODE_OUTPUT,
22  };
23  gpio_config(&ir_led_cfg);
24
25  ledc_timer_config_t timer_cfg = {
26      .speed_mode = LEDC_MODE,
27      .timer_num = LEDC_TIMER,
28      .duty_resolution = LEDC_DUTY_RES,
29      .freq_hz = LEDC_FREQUENCY,
30      .clk_cfg = LEDC_AUTO_CLK,
31  };
32  ledc_timer_config(&timer_cfg);
33
34  ledc_channel_config_t channel_cfg = {
35      .gpio_num = PWM_LED_GPIO,
36      .speed_mode = LEDC_MODE,
37      .channel = LEDC_CHANNEL,
38      .timer_sel = LEDC_TIMER,
39      .duty = 0,
40      .hpoint = 0,
41  };
42  ledc_channel_config(&channel_cfg);
43  }

```

Listing 2: ADC, GPIO, and LEDC configuration, called once from app_main before either task starts

Task A – SensorTask

```

1  static void sensor_task(void *pvParameters)
2  {
3      TickType_t last_wake = xTaskGetTickCount();
4
5      while (1) {
6          int raw = 0;
7          adc_oneshot_read(adc_handle, POT_ADC_CHANNEL, &raw);
8          g_pot_raw = raw;
9
10         bool detected = (gpio_get_level(IR_SENSOR_GPIO) == 0);
11         g_ir_state = detected;
12
13         ESP_LOGI(TAG_SENSOR, "pot_raw=%d  ir_state=%d  tick=%lu",
14             raw, detected, (unsigned long)xTaskGetTickCount());
15
16         vTaskDelayUntil(&last_wake, pdMS_TO_TICKS(200));
17     }
18 }

```

Listing 3: Reads the potentiometer and IR sensor into shared globals every 200 ms

Task B – OutputTask

```

1  static void output_task(void *pvParameters)
2  {
3      TickType_t last_wake = xTaskGetTickCount();
4
5      while (1) {

```

```

6      int raw = g_pot_raw;
7      bool ir = g_ir_state;
8
9      int duty = raw >> 2;
10     g_pwm_duty = duty;
11
12     ledc_set_duty(LEDC_MODE, LEDC_CHANNEL, duty);
13     ledc_update_duty(LEDC_MODE, LEDC_CHANNEL);
14
15     gpio_set_level(IR_LED_GPIO, ir ? 1 : 0);
16
17     ESP_LOGI(TAG_OUTPUT, "duty=%d  ir_led=%d  tick=%lu",
18                duty, ir, (unsigned long)xTaskGetTickCount());
19
20     vTaskDelayUntil(&last_wake, pdMS_TO_TICKS(100));
21 }
22 }

```

Listing 4: Reads the shared globals and drives PWM duty and the IR LED every 100 ms

app_main – task creation and housekeeping

```

1 void app_main(void)
2 {
3     ESP_LOGI(TAG_MAIN, "Task 3.6 starting - init peripherals");
4     init_peripherals();
5
6     xTaskCreate(sensor_task, "SensorTask", 4096, NULL, 3, NULL);
7     xTaskCreate(output_task, "OutputTask", 4096, NULL, 4, NULL);
8
9     ESP_LOGI(TAG_MAIN, "Both tasks created - scheduler will interleave them");
10
11     char stats_buf[1024];
12     while (1) {
13         vTaskDelay(pdMS_TO_TICKS(5000));
14 #if CONFIG_FREERTOS_GENERATE_RUN_TIME_STATS
15         vTaskGetRunTimeStats(stats_buf);
16         ESP_LOGI(TAG_MAIN, "\nTask\t\tAbs Time\t%% Time\n%s", stats_buf);
17 #endif
18     }
19 }

```

Listing 5: Both tasks are created once; app_main then only reports runtime stats

6. Task Scheduling & Behavior

Both `SensorTask` and `OutputTask` are self-contained: each owns its own infinite loop and its own `vTaskDelayUntil` call, so neither task ever busy-waits and neither depends on the other's loop timing to make progress. `vTaskDelayUntil` (rather than `vTaskDelay`) was used in both tasks so that each task's period is measured from its own last wake time rather than from when it happened to finish its work, keeping the 200 ms and 100 ms periods stable even as logging and driver-call durations vary slightly between iterations.

`OutputTask` is created with priority 4, one level above `SensorTask`'s priority 3, matching the requirement that the output-driving task should preempt the sensor-reading task rather than the other way around: a stale LED update matters more to the observable behavior of the system than a slightly delayed sensor sample. In practice, at these periods and on hardware this simple, both tasks spend the overwhelming majority of their time *Blocked* inside `vTaskDelayUntil`, so the priority difference rarely causes visible preemption on its own – its effect would become apparent under real contention, e.g. if a third, lower-priority task were added that could otherwise starve `OutputTask`.

```
I (264) APP_MAIN: Task 3.7 starting - init peripherals
I (264) SENSOR_TASK: pot_raw=4095 ir_state=0 tick=1
I (264) OUTPUT_TASK: duty=1023 ir_led=0 tick=1
I (264) APP_MAIN: Both tasks created - scheduler will interleave them
I (364) OUTPUT_TASK: duty=1023 ir_led=0 tick=11
I (464) OUTPUT_TASK: duty=1023 ir_led=0 tick=21
I (464) SENSOR_TASK: pot_raw=4095 ir_state=0 tick=21
I (564) OUTPUT_TASK: duty=1023 ir_led=0 tick=31
I (664) OUTPUT_TASK: duty=1023 ir_led=0 tick=41
I (664) SENSOR_TASK: pot_raw=4095 ir_state=0 tick=41
I (764) OUTPUT_TASK: duty=1023 ir_led=0 tick=51
I (864) OUTPUT_TASK: duty=1023 ir_led=0 tick=61
I (864) SENSOR_TASK: pot_raw=4095 ir_state=0 tick=61
I (964) OUTPUT_TASK: duty=1023 ir_led=0 tick=71
```

Figure 3: Evidence of task interleaving from ESP_LOGI timestamps.

7. ADC-to-PWM Scaling

The ADC oneshot channel in this task is configured with `ADC_BITWIDTH_DEFAULT`, which resolves to the ESP32's native 12-bit range (0–4095). The LEDC timer is configured for 10-bit duty resolution (0–1023). Rather than the explicit linear scale used in the previous task, the conversion here is a plain right shift:

$$\text{duty} = \text{raw} \gg 2 = \left\lfloor \frac{\text{raw}}{4} \right\rfloor$$

Since $4095/4 \approx 1023$, a two-bit right shift maps the 12-bit ADC range onto the 10-bit LEDC range almost exactly, at the cost of the small quantization error inherent to using a power-of-two shift instead of the exact $\times 1023/4095$ ratio – negligible for a brightness control where the eye cannot resolve single LSB steps at this resolution.

8. Debugging Journal & Observed Data-Sharing Hazard

The naive global-variable hand-off

This task deliberately shares data between `SensorTask` and `OutputTask` through three plain `volatile` globals instead of a queue, in order to surface – not fix – a real synchronization hazard.

What is actually safe here. Each individual global is a single machine word (`int`) or a single byte (`bool`) on the ESP32's 32-bit Xtensa core, and each is marked `volatile`. A lone read or write of one such variable will not be torn – the compiler will not split a 32-bit or 8-bit access into two visible half-writes. Marking the variables `volatile` additionally prevents the compiler from caching a stale value in a register across loop iterations, which matters here specifically because the two tasks run in different execution contexts.

What is not safe. `g_pot_raw` and `g_ir_state` are written together, once per `SensorTask` iteration, but they are two separate memory writes with no barrier or lock spanning both. `OutputTask` runs on its own independent 100 ms schedule and can be preempted or simply scheduled at any point relative to `SensorTask`'s writes. This means `OutputTask` can legally observe a combination that never existed as a single consistent snapshot in `SensorTask` – for example, reading an already-updated `g_pot_raw` together with a not-yet-updated `g_ir_state` from the previous cycle, because the two writes did not happen atomically as a pair. Separately, because the two tasks run at different periods (200 ms vs. 100 ms), `OutputTask` will also simply re-read and re-apply the exact same `g_pot_raw` value on some iterations, since no new sample arrived since the last read – a staleness issue rather than a tearing issue, but a data-sharing hazard either way.

Deliberately not fixed here. No mutex or queue is introduced in this task. This is intentional: Task 3.7 replaces this raw global hand-off with a proper `Queue`, and this task exists specifically to make the naive version's failure mode visible first.

9. Results & Verification

```
I (37464) SENSOR_TASK: pot_raw=2401 ir_state=1 tick=3721
I (37564) OUTPUT_TASK: duty=600 ir_led=1 tick=3731
I (37664) OUTPUT_TASK: duty=600 ir_led=1 tick=3741
I (37664) SENSOR_TASK: pot_raw=2417 ir_state=1 tick=3741
I (37764) OUTPUT_TASK: duty=604 ir_led=1 tick=3751
I (37864) OUTPUT_TASK: duty=604 ir_led=1 tick=3761
I (37864) SENSOR_TASK: pot_raw=2416 ir_state=0 tick=3761
I (37964) OUTPUT_TASK: duty=604 ir_led=0 tick=3771
I (38064) OUTPUT_TASK: duty=604 ir_led=0 tick=3781
I (38064) SENSOR_TASK: pot_raw=2411 ir_state=0 tick=3781
I (38164) OUTPUT_TASK: duty=602 ir_led=0 tick=3791
I (38264) OUTPUT_TASK: duty=602 ir_led=0 tick=3801
I (38264) SENSOR_TASK: pot_raw=2416 ir_state=0 tick=3801
```

Figure 4: `SensorTask` and `OutputTask` log output over a representative interval.

`OutputTask` was confirmed to apply PWM duty and IR-LED state independently of `SensorTask`'s own schedule: sweeping the potentiometer while alternately covering and uncovering the IR sensor produced brightness and digital-LED changes that tracked each input correctly, with the digital LED's response bounded by `SensorTask`'s 200ms sampling period plus up to one `OutputTask` cycle, rather than by any interaction between the two tasks.

10. Report Questions

10.1 The five FreeRTOS task states

FreeRTOS tasks move between five states: **Running** (actively executing on a core right now), **Ready** (able to run, waiting only for the scheduler to grant it a core), **Blocked** (waiting on a timeout, a delay, or an event such as a queue or semaphore, and not eligible to run until that condition clears), **Suspended** (explicitly removed from scheduling by `vTaskSuspend` until explicitly resumed), and **Deleted** (torn down, its resources reclaimed, no longer known to the scheduler). In this project, both `SensorTask` and `OutputTask` spend the overwhelming majority of their lifetime **Blocked** inside `vTaskDelayUntil`, moving briefly through **Ready** and **Running** once per period to take a reading or apply an output, then returning to **Blocked**. Neither task is ever **Suspended** or **Deleted** in this design.

10.2 What priority actually decides

Task priority only matters at the moment two or more tasks are simultaneously **Ready** to run – it decides which one of them the scheduler grants a core to first. It says nothing about how often a task runs, how important its data is, or how fast it executes; a low-priority task that is otherwise **Ready** will still run to completion (or until it blocks) whenever no higher-priority task is **Ready** to compete with it. Here, giving `OutputTask` priority 4 against `SensorTask`'s priority 3 only has an observable effect on the rare occasions both tasks become **Ready** in the same tick.

10.3 A stale or half-updated read, concretely

Because `g_pot_raw` and `g_ir_state` are written as two separate, unsynchronized statements inside `SensorTask`, `OutputTask` can be scheduled in the narrow window between those two writes. In that window it would read a freshly-updated `g_pot_raw` paired with the *previous* cycle's `g_ir_state` – a combination of values that was never true at any single instant inside `SensorTask`. This happens because nothing establishes the two writes (or the corresponding two reads) as a single atomic unit; each variable is individually safe from tearing, but the pair, as a pair, is not protected at all.

10.4 Why the dual-core architecture matters

The ESP32 exposes two cores (`PRO_CPU` and `APP_CPU`), and ESP-IDF's FreeRTOS port is symmetric multiprocessing (SMP)-capable: tasks created with a core affinity of `tskNO_AFFINITY` (the default used by `xTaskCreate()` here) may be scheduled onto *either* core independently, and are not guaranteed to stay pinned to one. That means `SensorTask` and `OutputTask` are not guaranteed to run on the same core, and in fact may genuinely execute simultaneously in real time rather than merely being time-sliced on one core. This makes the data-sharing hazard in Section strictly worse than a single-core interleaving argument alone would suggest: on a single core, at least the two writes inside `SensorTask`

cannot be physically simultaneous with a read in `OutputTask`; across two cores, a genuinely concurrent read and write are possible, which is exactly the class of hazard a queue or mutex is needed to close.

11. Design Notes – Efficient Choices

1. **`vTaskDelayUntil` over `vTaskDelay` for periodic tasks.** Anchoring each task's next wake time to its last scheduled wake time, rather than to when it happened to finish, keeps each task's period accurate regardless of small variation in per-iteration work.
2. **Priority reflects output freshness, not sensor importance.** `OutputTask` outranks `SensorTask` because a delayed physical output is the user-visible failure mode; the sensor read itself has no strict deadline.
3. **Internal pull-up used where the pin supports it.** Unlike GPIO34 in the previous task, GPIO5 is a standard GPIO with internal pull-up/pull-down support, so `GPIO_PULLUP_ENABLE` was used directly instead of relying solely on the sensor module's own output stage.
4. **Per-task TAG values.** Giving `SensorTask` and `OutputTask` distinct `ESP_LOG` tags makes it possible to grep or visually separate each task's timeline from the interleaved serial output without adding any extra logging infrastructure.
5. **Deliberately unsynchronized shared state.** Leaving the globals unprotected in this task, rather than jumping straight to a queue, keeps the actual hazard observable in the log output instead of hiding it behind a fix introduced before the problem was demonstrated.

12. Known Limitations & Considerations

- **Unsynchronized cross-variable state.** As detailed in Section , `g_pot_raw` and `g_ir_state` can be observed by `OutputTask` as an inconsistent pair; this is a known, accepted limitation of this specific task, not an oversight to be fixed here.
- **No core affinity pinning.** Both tasks use `tskNO_AFFINITY`, so their relative execution order across the two cores is not fully deterministic run to run.
- **Runtime statistics are conditionally compiled.** The `vTaskGetRunTimeStats` block only produces output when `CONFIG_FREERTOS_GENERATE_RUN_TIME_STATS` is enabled in `sdkconfig`; without it, `app_main`'s 5-second loop runs but logs nothing.

13. Bill of Materials

Item	Qty	Notes
ESP32 DevKit development board	1	Unchanged from previous task
LED	2	One PWM-driven (GPIO2), one digital on/off (GPIO4)
Current-limiting resistor	2	220–330 Ω , one per LED
IR obstacle sensor module	1	3-pin digital output (VCC, GND, OUT); internal pull-up now enabled on the ESP32 side
Potentiometer	1	Wiper on GPIO34 (ADC1 channel 6)
Breadboard	1	
Jumper wires	Assorted	
USB cable	1	Flashing and serial monitor
PC running VS Code + PlatformIO Core	1	ESP-IDF v5.x+ toolchain

14. Future Improvements

- **Task 3.7: replace the raw globals with a Queue.** A `QueueHandle_t` carrying a small struct (`{raw, ir_state}`) would close the cross-variable tearing hazard identified in Section by making the hand-off atomic by construction.
- **Pin core affinity explicitly**, once the timing relationship between `SensorTask` and `OutputTask` needs to be deterministic, rather than left to the SMP scheduler's default placement.
- **Add a mutex around the runtime-stats buffer** if a third task is later added that might also read `stats_buf`, even though `app_main` is currently its sole owner.
- **Interrupt-driven IR sensing**, carried over from the previous task's future-work list, remains applicable once ISR and queue mechanics are introduced.